



Universidad Centroamericana José Simeón Cañas
Departamento de Electrónica e Informática
Programación de Dispositivos Móviles, Ciclo 01-2026
Parcial 2 — RankeUCA, Primera versión: ¿dónde almorzamos?

1. Descripción

Todo empezó con un problema muy real: a la hora del almuerzo nunca sé dónde ir a comer en la U. Hay un montón de opciones y siempre termino dando vueltas sin decidirme. Entonces se me ocurrió una solución: ¿y si mis estudiantes votan por el mejor lugar y yo solo miro quién gana? Problema resuelto.

Después caí en cuenta de que la misma idea sirve para un montón de cosas: hoy es la comida, pero mañana podría ser el mejor lugar para parquear, el mejor lugar para hacer las horas sociales o cualquier cosa de la vida universitaria que valga la pena discutir. Así nació RankeUCA, una plataforma donde la comunidad universitaria vota y ve, en tiempo real, qué va ganando. En esta primera versión (v1) nos quedamos con la pregunta original: ¿dónde se almuerza más rico en la U?

Tu trabajo es construir la app Android de esta v1. La app le muestra a la persona los lugares disponibles, le deja votar por su favorito con un solo toque, y luego le enseña cómo va la votación. Detrás de todo hay una API que entrega los lugares y guarda cada voto; tu app es la que conversa con ella. La API está documentada y protegida con una API key, así que parte del reto es leer su documentación, registrarte y autenticar tus llamadas.

Historia de usuario

Como catedrático indeciso que nunca sabe dónde almorzar en la U,
Quiero que mis estudiantes voten por el mejor lugar para comer y poder ver qué lugar va ganando,
Para por fin dejar de dar vueltas a la hora del almuerzo y saber de una vez a dónde ir a comer rico.

Generalidades

Modalidad	Individual, en clase
Duración	3:30 – 5:30 PM
Tipo de evaluación	Parcial práctico
Tecnologías	Kotlin · Jetpack Compose · Ktor · MVVM
Entrega	Repositorio en GitHub + revisión oral
Defensa	Explicación del código en vivo

El proyecto base

El catedrático ya aprendió lo básico y se animó a dejar listo el proyecto base: configuró las librerías, el cliente de Ktor, Coil para las imágenes y dejó la primera pantalla conectada a la navegación. Hasta ahí llegó. Ahora te toca a vos: quiere que le ayudés con la implementación de la arquitectura MVVM, el patrón Repository y el consumo de la API de votación.

Cloná el proyecto base desde acá:

github.com/alvarogarcia1010/PDM_Course_RankeUca_Template

Aclaración importante: el consumo de la API debe hacerse con Ktor, que es la librería que vimos en clase y que ya viene configurada y lista en el proyecto base. Resolverlo con otra librería de red restará puntos en el criterio de consumo de la API.

La API que vas a consumir

La API ya está corriendo y completamente documentada. Toda la información que necesitás está en la documentación: los endpoints, cómo registrarte para obtener tu API key, cómo autenticar tus llamadas y la forma exacta de cada request y response. Tu primer paso del parcial es abrir ese enlace y leerlo.

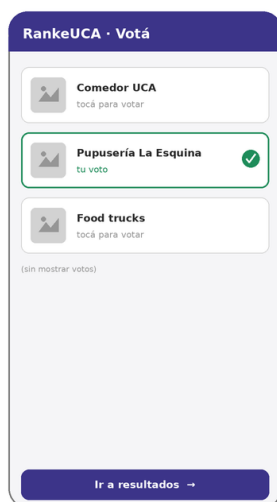
Documentación de la API (tu fuente oficial)

<https://qjcxdvfzyseuvezacxsd.supabase.co/functions/v1/rankeuca/docs>

Ojo: la API está protegida. No vas a poder traer los lugares ni votar hasta que te registres con tu carnet, obtengás tu API key y la envíes en cada llamada. La documentación te explica cómo; leela antes de escribir código.

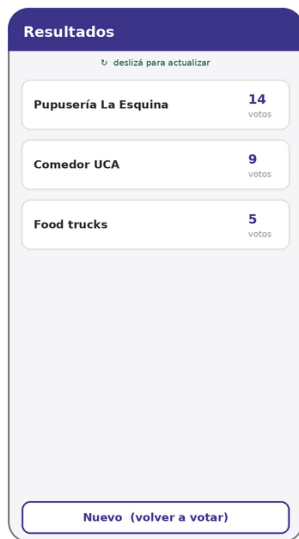
Funcionalidades

Cada pantalla se ve así. Los puntos al lado son lo que debe cumplir.



Pantalla 1: Votación

- Trae los lugares de la API y los muestra (imagen + nombre).
- NO muestra los votos: se vota a ciegas.
- Tocar un lugar = votar. Envía el voto de una vez, sin pantallas extra.
- Al votar bien: marca el lugar y habilita “Ir a resultados”.
- Maneja loading y error en cada request.



Pantalla 2: Resultados

- Vuelve a traer los lugares y muestra el nombre y el número de votos de cada uno.
- Ordenados de mayor a menor (el orden lo hacés vos en la app).
- Pull to refresh: deslizar hacia abajo recarga los votos.
- Maneja loading y error.
- Botón “Nuevo”: regresa a votar.

Para toda la app

- Toda request lleva tu API key en el header; si falta o es inválida (401), mostrar error, no pantalla en blanco.
- La navegación entre las dos pantallas funciona en ambos sentidos.
- Material Design 3.

Consulta permitida

Durante el parcial podés auxiliarte de tu propio portafolio (el código de tus ejercicios y prácticas anteriores) y de la documentación oficial de las tecnologías que usás. Hasta ahí.

Estrictamente prohibido

- Asistentes de inteligencia artificial (ChatGPT, Claude, Copilot, Gemini, etc.).
- Copiar código de otros compañeros.
- Repositorios de GitHub de terceros con soluciones similares.
- Presentaciones de clase, guías o talleres.

Sanción

El uso comprobado de IA o la copia entre compañeros será sancionado con cero en el parcial.

Defensa oral

Durante la revisión puedo pedirte que expliqués con tus propias palabras cómo funciona cualquier parte de tu código. Si no podés explicarlo, esa funcionalidad no cuenta.

Criterios de evaluación

La nota final es la suma ponderada de los criterios según su peso. Cada criterio se evalúa en uno de tres niveles: no cumple (0%), cumple a medias (50%) o cumple completamente (100%).

Criterio	No cumple (0%)	A medias (50%)	Cumple (100%)
1. Funcionalidad — 60%			
Autenticación y consumo de la API 20%	La API key no está registrada a su carnet (usó una ajena o inventada) o la autenticación no funciona: la app no logra traer ni enviar datos.	Autentica con la key de su carnet, pero una de las dos operaciones — traer los lugares o registrar el voto— no funciona como debería.	Se registra con su carnet, usa su API key en el header, el GET trae los lugares y el POST registra el voto correctamente.
Votación a ciegas 15%	No se puede votar.	Se puede votar pero se muestran los votos en esta pantalla, o tocar el lugar no registra el voto, o no se habilita el paso a resultados.	Muestra los lugares sin votos, tocar un lugar registra el voto, lo marca y habilita el acceso a resultados.
Resultados 15%	La pantalla de resultados no existe o no muestra datos.	Muestra los votos pero no los ordena de mayor a menor, o no implementa el pull to refresh.	Re-consulta la API, muestra nombre y votos ordenados de mayor a menor, y el pull to refresh recarga los datos.
Manejo de loading y error 10%	No hay indicación de carga ni manejo de errores.	Maneja uno de los dos (solo loading o solo error), o lo hace en una sola pantalla.	Muestra estado de carga y mensajes de error claros en las llamadas de ambas pantallas.
2. Patrón cumplido — 25%			
Arquitectura MVVM 10%	La lógica y el estado viven en la vista (composables); no hay ViewModel o no cumple su función.	Hay ViewModel, pero la vista todavía maneja parte del estado o contiene lógica propia.	El ViewModel expone el estado de la pantalla y la vista solo lo consume y lo dibuja, sin lógica de negocio.
Patrón Repository 10%	No hay repository; las llamadas a la API están sueltas en la vista o el ViewModel.	Hay repository pero sin interfaz (solo la clase concreta), o el ViewModel/vista llaman a Ktor directo en algunas partes.	El repository está definido con su interfaz e implementación; el ViewModel depende de la interfaz y toda llamada pasa por ahí.
Capa de API y DTOs 5%	No hay DTOs; se usa el mismo modelo para la API y la UI, o se parsea el JSON sin estructura.	Hay DTOs pero mezclados con los modelos de UI o de dominio, o no mapean bien la respuesta.	Los DTOs viven en la capa de API, separados de los modelos de UI, y mapean correctamente request y response.
3. La pregunta — 15%			
Defensa oral 15%	No puede explicar el código ni responder la pregunta del docente.	Explica el código de forma superficial o no responde con claridad la pregunta del docente.	Explica el código con sus propias palabras y responde correctamente la pregunta del docente.

Formato de entrega

- Entregá el enlace de un repositorio público de GitHub con el código del parcial.
- El repositorio debe tener un commit final que represente el estado entregado al momento de finalizar el tiempo.
- El proyecto debe compilar y correr.
- Después de la entrega, revisaremos el código juntos en el aula y darás una breve explicación oral, mostrando que comprendés lo que escribiste.

Éxitos. Confía en lo que ya practicaste.